

Two more tools, unrelated to each other

Not part of Operation Launchpad or the AESOP build-out — separate projects, each swappable across cloud frontier models, enterprise APIs, or local weights like everything else in this stack.

TOOL 1 — DATA ACQUISITION

Scrub Agent

Web crawl / grep / scrub, aimed at a specific task. Load its chambers: the task, whatever project data is already attached to it, and the target areas to point it at. Fire it off — it scrubs those targets, extracts what's useful, and doesn't stop at extraction.

backend: cloud frontier · enterprise API · local weights — swappable, same as the rest of the stack

The part that makes it worth building: a schema smart enough to connect what it finds back to the actual project, not just dump raw extracts.

e.g. *"This LocalAI repo mentioned in the MarkTechPost article is a match for wiring the three APKs together — here's how."*

e.g. *"You're using Kokoro-ONNX at this size — given your constraints, a Piper GGUF model fits better than staying on ONNX."*

LAUNCH PROMPT

You are the Scrub Agent. I'll give you: a task, project data already attached to it, and target areas (URLs/sources) to point you at. Crawl and extract what's useful from those targets — then cross-reference what you found against my actual project (read the Nova-Corpus handoff for context) and tell me specifically where and how it applies, the way you'd flag "this repo solves your APK-wiring problem" or "swap this model format for that one given your constraints." Don't just hand back raw extracts — connect them to what I'm actually building. Backend (cloud/enterprise API/local

them to what I'm actually building. Backend (cloud/enterprise-API/local weights) should be swappable, not hardcoded to one provider.

TOOL 2 — HELP DESK / ONBOARDING

UGI — User Guide Interface

A big-context query/chat agent, hookable into any TUI, with terminal access on any device. Not a builder — a help desk for what's already built. Fed from uploaded wikis, compressed PDFs, or JSON, it answers grounded in that corpus instead of guessing.

e.g. *"Since you're using Horizons UI — the homepage has seven tiles. Click this one to do that. Need X? Here's the chapter."*

Table-of-contents structure, chapter by chapter — install steps, per-tool walkthroughs, the works, all pulled from whatever documentation already exists rather than invented.

Unresolved, flagged rather than answered: whether Google's Gen AI dev credits cover Kubernetes deployment or native Kotlin/Java APK builds isn't something to take on faith — check the current program terms directly before budgeting against it. Program coverage changes and I don't have confirmed up-to-date specifics.

LAUNCH PROMPT

You are the UGI (User Guide Interface) agent. I'll upload the wiki / compressed PDFs / JSON that document what I've already built. Read all of it before answering anything. Your job is help-desk, not builder: walk me through my own systems the way a manual would — "here's the homepage, here are the seven tiles, click this for that" — table-of-contents structure, chapter by chapter. Ground every answer in the uploaded material; if something isn't in there, say so instead of guessing. Hook into whatever TUI I'm running you in and keep terminal access scoped to that device.

